

MODULE 04 · DOMAIN 2 - DATA INGESTION AND LOADING

COPY INTO

The idempotent SQL loader: syntax, the files it remembers, schema evolution, and the one behaviour every question turns on.

DOMAIN 2 of 7 EXAM WEIGHT 21% MODULE 04 of 36

COPY INTO keeps a record of every file it has loaded, and skips them next time. That is the whole feature. **Everything else is detail.**

The anatomy of the statement

CLAUSE	WHAT IT DOES
<code>COPY INTO my_table</code>	The target Delta table. It must already exist.
<code>FROM '/Volumes/...'</code>	Source location: S3, ADLS, ABFS, GCS or a UC volume
<code>FILEFORMAT = CSV</code>	The format of the source files
<code>FORMAT_OPTIONS (...)</code>	Passed to the Spark reader for that format
<code>COPY_OPTIONS (...)</code>	Controls COPY INTO itself

ON THE EXAM

Two option blocks that look alike and do different jobs. `FORMAT_OPTIONS` is about reading files. `COPY_OPTIONS` is about the command.

What idempotency actually guarantees

- ◆ **Files already loaded are skipped.** The record lives with the table, not in your scheduler.
- ◆ **This holds even if the file has been modified since it was loaded.** Same filename, still skipped.
- ◆ **The operation is retryable.** A failed run can simply be run again.
- ◆ **The same statement is safe on a schedule.** No date filter, no bookkeeping of your own.

ON THE EXAM

That second point is the single most testable detail about COPY INTO. Tracking is by file, not by content.

Default behaviour, and force

DEFAULT

force = false

- Previously loaded files are skipped
- Safe to schedule; safe to retry
- Corrections to an already-loaded file are ignored
- This is what you want almost always

COPY_OPTIONS ('FORCE' = 'TRUE')

Idempotency disabled

- Every matching file is loaded, seen before or not
- Use with FILES or PATTERN to target a subset
- On an unfiltered directory, it reloads everything
- Never leave it on in a scheduled job

THE TRAP

force does not mean 'try harder'. It means 'forget what you have already loaded'.

mergeSchema, in two places

- ◆ **In FORMAT_OPTIONS:** merge the schemas of the source files with each other while reading them.
- ◆ **In COPY_OPTIONS:** evolve the target table's schema to accept the incoming data.
- ◆ **Both default to false.** Without the COPY_OPTIONS one, a new source column causes a failure rather than a new column.
- ◆ **Schema evolution questions need the COPY_OPTIONS form.** That is the one that changes the table.

ON THE EXAM

Same word, two blocks, two jobs. FORMAT_OPTIONS reconciles files with each other; COPY_OPTIONS reconciles them with the table.

Loading into a schemaless placeholder table

STEP	STATEMENT
Create an empty shell	CREATE TABLE IF NOT EXISTS my_table;
Load and infer	COPY INTO my_table FROM '/path'
	FILEFORMAT = JSON
	FORMAT_OPTIONS ('mergeSchema' = 'true')
	COPY_OPTIONS ('mergeSchema' = 'true');

ON THE EXAM

Available from Databricks Runtime 11.3 LTS. The empty table is unusable until COPY INTO has populated it: INSERT INTO and MERGE INTO are not supported on it.

Choosing which files to load

OPTION	BEHAVIOUR	LIMIT
FILES	An explicit list of file names	1000 files maximum
PATTERN	A glob pattern matching the source directory	No fixed limit
Neither	Everything in the source location	
Both together	Not allowed	Mutually exclusive

ON THE EXAM

FILES and PATTERN cannot be used in the same statement. If an option shows both, it is wrong on syntax alone.

VALIDATE

- ◆ **Runs the load without writing anything**, so you can see what would happen.
- ◆ **The default validates all of the data** that would be loaded.
- ◆ **VALIDATE 15 ROWS** limits it to a given number of rows.
- ◆ **Returns a preview** of up to 50 rows.

ON THE EXAM

Useful for a first load against unfamiliar files, where you want to see the parse result before committing it to a table.

BY POSITION

- ◆ Matches source columns to target columns by ordinal position rather than by name.
- ◆ Type casting of matched columns happens automatically.
- ◆ Only supported for headerless CSV. FILEFORMAT must be CSV, with headers set to false.
- ◆ The alternative is naming columns explicitly in the COPY INTO clause.

ON THE EXAM

A narrow feature with a hard constraint. If the format is not headerless CSV, BY POSITION is not available.

What COPY INTO can read

FORMATS	SOURCES
CSV, JSON, XML	Amazon S3
Avro, ORC, Parquet	ADLS and ABFS
Text and binary files	Google Cloud Storage
	Unity Catalog volumes

ON THE EXAM

Note what is absent: message buses. COPY INTO reads files from object storage. Kafka and Kinesis are Structured Streaming's territory.

COPY INTO or Auto Loader?

COPY INTO

Thousands of files

- A single SQL statement, schedulable
- Directory listing only, to discover files
- Easy to reload a chosen subset with FILES or PATTERN
- No rescued data column; schema must be settable

AUTO LOADER

Millions of files

- A streaming source, cloudFiles
- Directory listing or file notification
- Harder to reprocess a specific subset
- Richer schema inference, evolution and rescued data

THE TIEBREAKER

Databricks' own line: thousands of files over time, use COPY INTO. Millions or more, use Auto Loader.

Four things COPY INTO cannot do

- ◆ **No rescued data column.** Records that fail to parse are not captured to one side for you.
- ◆ **Directory listing only.** No file notification mode, so discovery cost grows with directory size.
- ◆ **No message buses.** Files in object storage only.
- ◆ **It will not re-read a modified file.** Same name, already loaded, skipped.

ON THE EXAM

Each of these is a reason a question steers you to Auto Loader, streaming tables, or Structured Streaming instead.

A currency note

Worth knowing, because the documentation has started to shift

- ◆ **Databricks now recommends streaming tables** for SQL users wanting scalable, robust file ingestion.
- ◆ **Some documentation pages now label it 'COPY INTO (legacy)'.**
- ◆ **It remains fully supported**, and the exam guide names it as an explicit objective.
- ◆ **Learn it as taught here.** Streaming tables appear in module 13.

ON THE EXAM

Do not read 'legacy' as 'wrong answer'. The current exam guide names COPY INTO by name in Domain 2.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Scheduled nightly, no duplicates appear	Idempotency, on by default
A corrected file was re-uploaded but ignored	Same filename, already loaded, skipped
The table must gain a new source column	mergeSchema in COPY_OPTIONS
Reload one specific batch of files	FILES or PATTERN, with force
Millions of files over time	Auto Loader, not COPY INTO
Check the parse before writing anything	VALIDATE

ON THE EXAM

Six lines. The second one is the detail most candidates get wrong.

A nightly job runs COPY INTO against a folder of daily CSV exports. On Tuesday the team discovers that Monday's file, sales_2026_03_16.csv, contained corrupted amounts. The supplier corrects the file and re-uploads it under the same name.

The team re-runs the COPY INTO statement. The corrected amounts do not appear in the table, and the run reports that no files were loaded.

Why did nothing load?

- A The re-uploaded file has a modification timestamp that is out of order, so it was skipped
- B COPY INTO tracks files by name and had already loaded that file, so it skipped it despite the change
- C The target table's schema no longer matches the corrected file
- D COPY INTO only loads files created after the most recent successful load

◆ PAUSE AND DECIDE

Why did nothing load?

A Out-of-order modification timestamp

Plausible-sounding, but timestamps are not the mechanism. The file was skipped because of its identity, not its age.

C Schema mismatch

A schema mismatch produces an error, not a quiet report that zero files were loaded. The symptom does not fit.

B Already loaded, so skipped despite the modification

Correct. The documentation is explicit: already-loaded files are skipped even if they have been modified since loading.

D Only loads files created after the last load

Describes a timestamp watermark, which is how people imagine COPY INTO works. It tracks a set of files instead.

THE SIGNAL

Fix it with COPY_OPTIONS ('force' = 'true') plus FILES limited to that one file. Forcing the whole directory would reload everything.

An IoT platform writes one small JSON file per device per minute into cloud storage. There are 9,000 devices. The directory already holds over 200 million files and grows by roughly 13 million a month.

A team proposes a nightly COPY INTO statement against the directory to load the previous day's readings into bronze.

What is the strongest objection?

- A COPY INTO cannot read JSON, so the format is wrong
- B COPY INTO discovers files by directory listing, which does not scale to hundreds of millions of files
- C COPY INTO would create duplicates because the files arrive every minute
- D COPY INTO cannot write to a bronze table because bronze must be append-only

◆ PAUSE AND DECIDE

What is the strongest objection?

- A

 COPY INTO cannot read JSON
False. JSON is one of its eight supported formats, alongside CSV, XML, Avro, ORC, Parquet, text and binary.
- B

 Directory listing does not scale to this many files
Correct. COPY INTO only discovers files by listing. Databricks draws the line at thousands of files; this is 200 million.
- C

 It would create duplicates
Backwards. Idempotency is on by default, so arrival frequency does not cause duplicates. This is the opposite of the truth.
- D

 Cannot write to an append-only bronze table
Invented restriction. COPY INTO appends, which is precisely what a bronze table wants.

THE SIGNAL

Two numbers decide it: thousands of files is COPY INTO's range, millions is Auto Loader's. This scenario is 200 million.

An engineer notices that some days a supplier delivers files late, after the nightly load has run. Worried that data is being missed, she edits the scheduled statement to add `COPY_OPTIONS ('force' = 'true')`.

The next morning the table contains four times as many rows as expected. The row count roughly matches the total number of records ever delivered.

What happened?

- ☐ A force made COPY INTO retry failed files, and the retries were written twice
- ☐ B force disabled idempotency, so the entire directory was reloaded on top of the existing data
- ☐ C The late-arriving files had duplicate contents, and force preserved them
- ☐ D force switched the write mode from append to overwrite, doubling the data

◆ PAUSE AND DECIDE

What happened?

- A

 force retried failed files
force has nothing to do with retries. COPY INTO is retryable by default precisely because idempotency is on.
- B

 Idempotency disabled, so the whole directory reloaded
Correct. force means load everything matching the source, seen before or not. Unfiltered, that is the entire history.
- C

 Late files had duplicate contents
Blames the data. The row count matching everything ever delivered points at the command, not at a few late files.
- D

 force switched append to overwrite
Overwrite would replace the data, giving roughly the expected count. The table grew instead, so it appended.

THE SIGNAL

The engineer's actual problem was real. Late files are picked up on the next run anyway, because they have not been loaded yet.

A team runs an Auto Loader stream that continuously ingests files from a landing directory into a bronze table. A supplier reports that 40 files delivered last week were malformed, and re-uploads corrected versions under new names.

The team needs those 40 corrected files loaded, without stopping the running stream and without reprocessing anything else.

What is the cleanest approach?

- ☐ A Stop the stream, delete its checkpoint, and restart it to reprocess the directory
- ☐ B Run COPY INTO with a FILES list naming the 40 corrected files, alongside the running stream
- ☐ C Run COPY INTO with force = true against the whole landing directory
- ☐ D Wait for the Auto Loader stream to detect the corrected files on its next pass

◆ PAUSE AND DECIDE

What is the cleanest approach?

- A

 Delete the checkpoint and restart
Deleting the checkpoint discards all record of what was processed, so the stream reingests the entire directory. Duplicates on a large scale.
- B

 COPY INTO with a FILES list, alongside the stream
Correct. Databricks documents exactly this: COPY INTO can reload a chosen subset while an Auto Loader stream runs simultaneously.
- C

 COPY INTO with force on the whole directory
Loads all 40, and everything else as well. It meets one requirement and violates the other.
- D

 Wait for Auto Loader to find them
Re-read the setup: the corrections were uploaded under new names, so Auto Loader will find them. But the question asks for the cleanest approach to a stated need, and waiting is not an approach.

THE SIGNAL

COPY INTO's weakness at scale is its strength here. FILES gives surgical control that a stream's checkpoint cannot.

COPY INTO remembers **files**, not contents, and not timestamps. Nearly every question about it is testing whether you know which.